Material de apoio: prova prática de POO

Nov 9, 2026 · @Warley

Os oito itens do checklist prontos: cada código abaixo compilou e rodou com entradas de teste, e está também no package `apoio` do projeto do Eclipse.

Item 1: esqueleto de classe modelo

Ponto de partida para qualquer questão de classe: troque `Aluno` pelo nome do enunciado e mantenha a ordem constantes, atributos, construtor, get/set, comportamento, `toString`. Arquivo: `apoio/Aluno.java`.

```
package apoio;

public class Aluno {

    // 1) Constantes: limites e valores válidos (public static final,
MAIÚSCULAS)
    public static final double NOTA_MIN = 0;
    public static final double NOTA_MAX = 10;

    // 2) Atributos: sempre private (encapsulamento)
    private String nome;
    private int idade;
    private double nota;

    // 3) Construtor: chama os set, assim a validação vale já na criação
    public Aluno(String nome, int idade, double nota) {
        setNome(nome);
        setIdade(idade);
        setNota(nota);
    }

    // Construtor vazio (opcional): objeto nasce com valores padrão
    public Aluno() {
        this.nome = "Sem nome";
        this.idade = 0;
        this.nota = NOTA_MIN;
    }

    // 4) get e set: a regra do enunciado fica dentro do set
    public String getNome() {
        return nome;
    }
```

```
public void setNome(String nome) {
    if (nome != null && !nome.trim().isEmpty()) {
        this.nome = nome;
    } else {
        System.out.println("Nome inválido.");
    }
}

public int getIdade() {
    return idade;
}

public void setIdade(int idade) {
    if (idade >= 0 && idade <= 150) {
        this.idade = idade;
    } else {
        System.out.println("Idade inválida: " + idade);
    }
}

public double getNota() {
    return nota;
}

public void setNota(double nota) {
    if (nota >= NOTA_MIN && nota <= NOTA_MAX) {
        this.nota = nota;
    } else {
        System.out.println("Nota inválida: " + nota);
    }
}

// 5) Métodos de comportamento (o que o objeto FAZ)
public String situacao() {
    if (nota >= 7) {
        return "Aprovado";
    } else if (nota >= 5) {
        return "Recuperação";
    } else {
        return "Reprovado";
    }
}

// 6) toString: texto que aparece ao imprimir o objeto
@Override
```

```
    public String toString() {
        return "Nome: " + nome + "\nIdade: " + idade
             + "\nNota: " + nota + "\nSituação: " + situacao();
    }
}
```

Para adaptar: estado com opções fixas (lâmpada acesa/apagada) vira constante `String` e o `set` aceita só essas opções com `equals`; um método que só funciona em certa condição (TV ligada) começa com `if (!ligada) { ... return; }`.

Item 2: esqueleto de classe de uso com JOptionPane

A classe que roda: lê texto, converte com `Integer.parseInt` e `Double.parseDouble`, cria o objeto e mostra. Arquivo: `apoio/UsaAluno.java`.

```
package apoio;

import javax.swing.JOptionPane;

public class UsaAluno {

    public static void main(String[] args) {
        // showInputDialog SEMPRE devolve String
        String nome = JOptionPane.showInputDialog("Nome:");

        // String -> int
        int idade = Integer.parseInt(JOptionPane.showInputDialog("Idade:"));

        // String -> double (digitar com PONTO: 7.5)
        double nota =
Double.parseDouble(JOptionPane.showInputDialog("Nota:"));

        // Cria o objeto (o construtor chama os set e valida)
        Aluno a = new Aluno(nome, idade, nota);

        // Mostra: na janela e no Console
        JOptionPane.showMessageDialog(null, a);   // usa o toString
        System.out.println(a);

        // Usando get e set depois de criado
        a.setNota(11);                             // inválida: mostra
mensagem e não muda
        System.out.println("Nota continua: " + a.getNota());
    }
}
```

Com as entradas `Maria`, `20` e `8.5`, o Console mostra:

```
Nome: Maria
Idade: 20
Nota: 8.5
Situação: Aprovado
Nota inválida: 11.0
Nota continua: 8.5
```

Item 3: validação com laço e try/catch

Quatro métodos `static` prontos que pedem o dado de novo até ser válido; chame de qualquer classe com `Validacao.lerInteiro("Idade:", 0, 150)` (em outro package, acrescente `import apoio.Validacao;`). Arquivo: `apoio/Validacao.java`.

```
package apoio;

import javax.swing.JOptionPane;

public class Validacao {

    // Lê um inteiro entre min e max; repete se digitar texto ou valor fora
da faixa
    public static int lerInteiro(String mensagem, int min, int max) {
        while (true) {
            try {
                int valor =
Integer.parseInt(JOptionPane.showInputDialog(mensagem));
                if (valor >= min && valor <= max) {
                    return valor;                    // válido: sai do laço
                }
                JOptionPane.showMessageDialog(null, "Digite um valor entre "
+ min + " e " + max);
            } catch (NumberFormatException e) {      // digitou texto ou
cancelou
                JOptionPane.showMessageDialog(null, "Digite apenas números
inteiros.");
            }
        }
    }

    // Lê um double maior que zero
    public static double lerPositivo(String mensagem) {
        double valor = -1;
        boolean valido = false;
```

```
        do {
            try {
                valor =
Double.parseDouble(JOptionPane.showInputDialog(mensagem));
                if (valor > 0) {
                    valido = true;
                } else {
                    JOptionPane.showMessageDialog(null, "O valor precisa ser
maior que zero.");
                }
            } catch (NumberFormatException | NullPointerException e) {
                JOptionPane.showMessageDialog(null, "Valor inválido. Use
ponto: 2.5");
            }
        } while (!valido);
        return valor;
    }

    // Lê um texto com tamanho mínimo
    public static String lerTexto(String mensagem, int tamanhoMinimo) {
        String texto = JOptionPane.showInputDialog(mensagem);
        while (texto == null || texto.trim().length() < tamanhoMinimo) {
            JOptionPane.showMessageDialog(null, "Mínimo de " + tamanhoMinimo
+ " caracteres.");
            texto = JOptionPane.showInputDialog(mensagem);
        }
        return texto;
    }

    // Lê uma opção de uma lista fixa (ex.: "Masculino" ou "Feminino")
    public static String lerOpcao(String mensagem, String op1, String op2) {
        String texto = JOptionPane.showInputDialog(mensagem);
        while (texto == null || !(texto.equalsIgnoreCase(op1) ||
texto.equalsIgnoreCase(op2))) {
            JOptionPane.showMessageDialog(null, "Digite " + op1 + " ou " +
op2);
            texto = JOptionPane.showInputDialog(mensagem);
        }
        return texto;
    }

    // Demonstração (mesmo enunciado da Lista 4, ex. 3)
    public static void main(String[] args) {
        String nome = lerTexto("Nome (mais de 5 letras):", 6);
        int idade = lerInteiro("Idade (0 a 150):", 0, 150);
        double salario = lerPositivo("Salário:");
```

```
        String sexo = lerOpcao("Sexo (Masculino/Feminino):", "Masculino",
"Feminino");

        JOptionPane.showMessageDialog(null, "Nome: " + nome + "\nIdade: " +
idade
                + "\nSalário: " + salario + "\nSexo: " + sexo);
    }
}
```

Testado com entradas erradas de propósito: `Ana` (curto), `abc` e `200` na idade, `-5` e `x` no salário, `outro` no sexo. Cada uma mostrou a mensagem e pediu de novo; com `Ana Paula`, `30`, `2500.50` e `feminino` o programa terminou.

Item 4: um código de cada estrutura

Quatro padrões que aparecem dentro de quase toda questão: faixas com `if`, laço com acumulador, laço com parada e menu com `switch`.

4a. If / else if por faixas (`apoio/Faixas.java`, padrão da Lista 2). Teste as faixas sempre na mesma direção: o primeiro `if` verdadeiro ganha e o resto é pulado.

```
package apoio;

import javax.swing.JOptionPane;

public class Faixas {

    public static void main(String[] args) {
        double salario =
Double.parseDouble(JOptionPane.showInputDialog("Salário:"));
        double percentual;

        if (salario < 500) {
            percentual = 20;
        } else if (salario < 1000) {      // aqui já se sabe que é >= 500
            percentual = 15;
        } else if (salario < 2000) {
            percentual = 10;
        } else {
            percentual = 0;
        }

        double novo = salario + salario * percentual / 100;
        JOptionPane.showMessageDialog(null, "Aumento: " + percentual +
"%\nNovo salário: " + novo);
```

```
        // Par ou ímpar (resto da divisão)
        int n = Integer.parseInt(JOptionPane.showInputDialog("Número
inteiro:"));
        if (n % 2 == 0) {
            JOptionPane.showMessageDialog(null, n + " é par");
        } else {
            JOptionPane.showMessageDialog(null, n + " é ímpar");
        }

        // Operadores: && (e), || (ou), ! (não), == (igual), != (diferente)
        // String se compara com equals, NUNCA com ==
    }
}
```

Com salário `1500`: aumento 10.0% e novo salário 1650.0.

4b. Laço com acumulador e contador (`apoio/Acumulador.java`, padrão da Lista 3). Use `for` quando sabe quantas voltas.

```
package apoio;

import javax.swing.JOptionPane;

public class Acumulador {

    public static void main(String[] args) {
        int quantidade = 5;
        double soma = 0;          // acumulador começa em 0
        int pares = 0;            // contadores começam em 0
        int impares = 0;

        for (int i = 1; i <= quantidade; i++) {
            int valor = Integer.parseInt(JOptionPane.showInputDialog("Valor "
+ i + " de " + quantidade + ":"));
            soma += valor;
            if (valor % 2 == 0) {
                pares++;
            } else {
                impares++;
            }
        }

        double media = soma / quantidade;
        JOptionPane.showMessageDialog(null, "Soma: " + soma + "\nMédia: " +
media
```

```
                + "\nPares: " + pares + "\nÍmpares: " + impares);

        // Somatório sem entrada: pares de 1 a 500
        int somaPares = 0;
        for (int i = 2; i <= 500; i += 2) {
            somaPares += i;
        }
        System.out.println("Soma dos pares de 1 a 500: " + somaPares);
    }
}
```

Com `1, 2, 3, 4, 5`: soma 15.0, média 3.0, 2 pares, 3 ímpares; soma dos pares de 1 a 500 = 62750.

4c. Laço com condição de parada (`apoio/LacoParada.java`, padrão da Lista 3, ex. 6 e 7). Use `while` quando não sabe quantas voltas.

```
package apoio;

import javax.swing.JOptionPane;

public class LacoParada {

    public static void main(String[] args) {
        int valor = Integer.parseInt(JOptionPane.showInputDialog("Valor
(negativo para parar):"));

        if (valor < 0) {
            JOptionPane.showMessageDialog(null, "Nenhum valor informado.");
            return;
        }

        int maior = valor;        // o primeiro valor é o maior e o menor até
agora
        int menor = valor;
        int contador = 0;

        while (valor >= 0) {
            contador++;
            if (valor > maior) {
                maior = valor;
            }
            if (valor < menor) {
                menor = valor;
            }
            valor = Integer.parseInt(JOptionPane.showInputDialog("Valor
```

```
(negativo para parar):"));
        }

        JOptionPane.showMessageDialog(null, "Valores lidos: " + contador
                + "\nMaior: " + maior + "\nMenor: " + menor);

        // Variante "deseja continuar?" (Lista 3, ex. 6)
        double total = 0;
        String resposta;
        do {
            double area =
Double.parseDouble(JOptionPane.showInputDialog("Área do cômodo:"));
            total += area;
            resposta = JOptionPane.showInputDialog("Deseja continuar?
(SIM/NÃO)");
        } while (resposta != null && resposta.equalsIgnoreCase("SIM"));
        JOptionPane.showMessageDialog(null, "Área total: " + total);
    }
}
```

Com `5, 9, 2, -1`: 3 valores, maior 9, menor 2. Com áreas `10` e `15.5`: total 25.5.

4d. Menu com switch (`apoio/Menu.java`, padrão da Lista 6, ex. 5). O `do-while` mantém o menu até escolher 0; não esqueça o `break` em cada `case`.

```
package apoio;

import javax.swing.JOptionPane;

public class Menu {

    public static void main(String[] args) {
        int opcao;
        do {
            opcao = Integer.parseInt(JOptionPane.showInputDialog(
                    "1 - Soma\n2 - Subtração\n3 - Multiplicação\n4 -
Divisão\n"
                  + "5 - Potência\n6 - Raiz quadrada\n0 - Sair"));

            double a, b;
            switch (opcao) {
                case 1:
                    a =
Double.parseDouble(JOptionPane.showInputDialog("Primeiro número:"));
                    b =
Double.parseDouble(JOptionPane.showInputDialog("Segundo número:"));
```

```
                    JOptionPane.showMessageDialog(null, "Resultado: " + (a +
b));
                    break;
                case 2:
                    a =
Double.parseDouble(JOptionPane.showInputDialog("Primeiro número:"));
                    b =
Double.parseDouble(JOptionPane.showInputDialog("Segundo número:"));
                    JOptionPane.showMessageDialog(null, "Resultado: " + (a -
b));
                    break;
                case 3:
                    a =
Double.parseDouble(JOptionPane.showInputDialog("Primeiro número:"));
                    b =
Double.parseDouble(JOptionPane.showInputDialog("Segundo número:"));
                    JOptionPane.showMessageDialog(null, "Resultado: " + (a *
b));
                    break;
                case 4:
                    a =
Double.parseDouble(JOptionPane.showInputDialog("Primeiro número:"));
                    b =
Double.parseDouble(JOptionPane.showInputDialog("Segundo número:"));
                    if (b == 0) {
                        JOptionPane.showMessageDialog(null, "Não existe
divisão por zero.");
                    } else {
                        JOptionPane.showMessageDialog(null, "Resultado: " +
(a / b));
                    }
                    break;
                case 5:
                    a =
Double.parseDouble(JOptionPane.showInputDialog("Base:"));
                    b =
Double.parseDouble(JOptionPane.showInputDialog("Expoente:"));
                    JOptionPane.showMessageDialog(null, "Resultado: " +
Math.pow(a, b));
                    break;
                case 6:
                    a =
Double.parseDouble(JOptionPane.showInputDialog("Número:"));
                    JOptionPane.showMessageDialog(null, "Resultado: " +
Math.sqrt(a));
                    break;
```

```
                case 0:
                    JOptionPane.showMessageDialog(null, "Fim do programa.");
                    break;
                default:
                    JOptionPane.showMessageDialog(null, "Opção inválida.");
            }
        } while (opcao != 0);
    }
}
```

Item 5: cola de String

Índices começam em 0 (`"FATEC"`: F=0, A=1, T=2, E=3, C=4). Valores obtidos rodando `apoio/ColaString.java` com a frase `s = "Hoje estamos na FATEC"`.

Método	Exemplo	Resultado
`length()`	`s.length()`	`21` (conta espaços)
`charAt(i)`	`s.charAt(0)`	`H`
`substring(ini, fim)`	`s.substring(0, 4)`	`Hoje` (o fim não entra)
`substring(ini)`	`s.substring(16)`	`FATEC` (até o fim)
`indexOf(texto)`	`s.indexOf("na")`	`13` (`-1` se não achar)
`contains(texto)`	`s.contains("FATEC")`	`true`
`toUpperCase()`	`s.toUpperCase()`	`HOJE ESTAMOS NA FATEC`
`toLowerCase()`	`s.toLowerCase()`	`hoje estamos na fatec`
`replace(a, b)`	`s.replace(" ", "")`	`HojeestamosnaFATEC`
`trim()`	`" oi ".trim()`	`oi`
`equals(t)`	`"java".equals("Java")`	`false`
`equalsIgnoreCase(t)`	`"java".equalsIgnoreCase("Java")`	`true`
`startsWith(t)`	`s.startsWith("Hoje")`	`true`
`isEmpty()`	`"".isEmpty()`	`true`
`split(sep)`	`s.split(" ")`	vetor `[Hoje, estamos, na, FATEC]`
`Integer.parseInt`	`Integer.parseInt("42")`	`42` (int)

Método	Exemplo	Resultado
`Double.parseDouble`	`Double.parseDouble("2.5")`	`2.5` (double)
`String.valueOf`	`String.valueOf(42)`	`"42"` (String)

Os quatro algoritmos de String que mais caem, prontos:

```
// Separar palavras (uma por linha)
String[] palavras = s.split(" ");
for (String p : palavras) {
    System.out.println(p);
}

// Inverter: percorre do fim para o começo ("Java" -> "avaJ")
String palavra = "Java";
String invertida = "";
for (int i = palavra.length() - 1; i >= 0; i--) {
    invertida += palavra.charAt(i);
}

// Contar vogais ("Hoje estamos na FATEC" -> 8)
int vogais = 0;
String minuscula = s.toLowerCase();
for (int i = 0; i < minuscula.length(); i++) {
    char c = minuscula.charAt(i);
    if ("aeiouáéíóúâêôãõ".indexOf(c) != -1) {
        vogais++;
    }
}

// Escada: J / Ja / Jav / Java
for (int i = 1; i <= palavra.length(); i++) {
    System.out.println(palavra.substring(0, i));
}
```

Item 6: cola de Math e Random

`Math` não precisa de `import` nem de `new`; `Random` precisa de `import java.util.Random;`. Valores obtidos rodando `apoio/ColaMath.java`.

Método	Exemplo	Resultado
Potência	`Math.pow(2, 3)`	`8.0`

Método	Exemplo	Resultado
Raiz quadrada	`Math.sqrt(25)`	`5.0`
Raiz cúbica	`Math.cbrt(27)`	`3.0`
Valor absoluto	`Math.abs(-7)`	`7`
Maior / menor	`Math.max(4, 9)` / `Math.min(4, 9)`	`9` / `4`
Arredondar	`Math.round(2.5)`	`3`
Arredondar para cima	`Math.ceil(2.1)`	`3.0`
Arredondar para baixo	`Math.floor(2.9)`	`2.0`
Pi	`Math.PI`	`3.141592653589793`
Sorteio 1 a 6	`(int) (Math.random() * 6) + 1`	um inteiro de 1 a 6
Sorteio 0 a 99	`new Random().nextInt(100)`	um inteiro de 0 a 99
Sorteio 1 a 10	`new Random().nextInt(10) + 1`	um inteiro de 1 a 10
2 casas decimais	`String.format("%.2f", 1234.5678)`	`1234,57` no Windows em português (`1234.57` em inglês)

Dois usos prontos que já caíram nas listas:

```
import java.util.Random;

// Senha numérica de 6 dígitos (Lista 6, ex. 2)
Random r = new Random();
String senha = "";
for (int i = 0; i < 6; i++) {
    senha += r.nextInt(10);
}

// Separar dígitos de um número de 4 dígitos (Lista 6, ex. 3): 4571
int numero = 4571;
int milhar  = numero / 1000;        // 4
int centena = numero / 100 % 10;    // 5
int dezena  = numero / 10 % 10;     // 7
int unidade = numero % 10;          // 1
```

Itens 7 e 8: projeto com as listas e README

O arquivo `POO_Projeto_com_Apoio.zip` tem o projeto `POO_Exercicios` completo: as 9 listas e a Granja, mais o package `apoio` com todos os códigos acima. O `README.md` dele foi atualizado com o mapa abaixo.

Package	Assunto	Use na prova para
`apoio`	Material de apoio (itens 1 a 6)	Copiar o esqueleto e adaptar
`lista2`	Condicionais	Faixas, par/ímpar, nota e média
`lista3`	Laços	Somatórios, contadores, maior/menor
`lista4`	Exceções e validação	Pedir de novo até ser válido
`lista5`	String	Inverter, vogais, split
`lista6`	Math	Sorteio, potência, menu
`lista7`	Classes com estado	Televisao, Garrafa
`lista8`	Métodos	`void` e com retorno
`lista9`	Encapsulamento	Lampada, Produtos, Tv
`granja`	Encapsulamento com cálculo	Galinha

Como colocar no Eclipse:

1. Se o projeto `POO_Exercicios` antigo já estiver no Eclipse, clique nele, aperte **F2** e renomeie para `POO_Antigo`. Assim o novo não dá conflito de nome e nada do que você escreveu no antigo se perde.
2. **File > Import > General > Existing Projects into Workspace > Next**.
3. Marque **Select archive file**, escolha `POO_Projeto_com_Apoio.zip` e clique em **Finish**.
4. Abra **src > apoio** e rode `UsaAluno` (**Run As > Java Application**) para conferir.

Na prova: crie a classe nova no package do exercício, copie o corpo de `Aluno` e `UsaAluno`, troque os nomes com **Alt+Shift+R** (renomeia em todo o arquivo) e ajuste as regras do `set`. Leve também o zip num pendrive e confira antes que ele abre no Eclipse do laboratório.